

Group Relative Policy Optimization

pig7selene

September 5, 2026

Abstract

Group Relative Policy Optimization (GRPO) is an online policy-optimization method that compares several responses sampled for the same prompt. Rather than training a separate Value Model to estimate an advantage, it uses the group’s reward distribution to construct a relative, advantage-like signal for each response. This chapter develops group-normalized rewards, the PPO-style clipped objective and reference-model constraint retained by GRPO, and the resulting rollout loop. It distinguishes GRPO from PPO and Direct Preference Optimization, explains why verifiable tasks are a natural setting for group-relative learning, and identifies the sampling, reward, and stability conditions on which the method depends.

1 Introduction

Chapter 16 formulated an autoregressive language model as a policy and developed PPO as an actor-critic method for optimizing rollout rewards. Chapter 17 then presented DPO, which uses fixed preference pairs in an offline likelihood objective. Group Relative Policy Optimization (GRPO) is closer to PPO in its dataflow: it samples new responses from a behavior-policy snapshot, scores them, and applies a constrained policy update. Its distinctive change is the baseline used to assign credit.

In the common outcome-reward setting, a completed response receives one scalar score. PPO usually learns a Value Model to estimate whether each token action led to a better or worse return than expected from its prefix. GRPO instead samples several responses for the **same** prompt and asks a simpler comparative question: which sampled responses did better than the other responses in this group? The group mean provides a prompt-local baseline, and the group spread provides a local scale. GRPO was introduced in DeepSeekMath as a PPO variant that forgoes the critic model in favor of group-relative rewards [1].

The method is therefore neither ordinary supervised learning nor offline preference optimization. Its central loop is

$$\text{prompt} \rightarrow \text{group Rollouts} \rightarrow \text{score} + \text{normalize} \rightarrow \text{clipped update} + \text{KL} \rightarrow \text{repeat}. \quad (1)$$

This chapter develops that loop without treating it as a full account of Reasoning RL. In particular, it uses verifiable rewards to explain why group comparisons can be useful, but leaves the design of Outcome Reward, Process Reward, and verifier systems to the next chapter.

2 Group-Based Rollouts and Relative Rewards

Let x be a prompt drawn from a declared distribution $\mathcal{D}_{\text{prompt}}$. At one rollout step, freeze the current behavior policy as π_{old} and sample a group of G responses,

$$y_i = (a_{i,0}, \dots, a_{i,T_i-1}) \sim \pi_{\text{old}}(\cdot \mid x), \quad i \in \{1, \dots, G\}. \quad (2)$$

The responses in Equation 2 share the prompt and sampling policy revision, but they may have different lengths, stop reasons, and reward outcomes. A reward procedure assigns each response a scalar r_i . It may be a learned Reward Model score, a rule-based verifier, a program-execution result, or a declared combination of such signals. The score is an **absolute reward** in the sense that the reward procedure evaluates each response independently; it need not be a probability or a calibrated utility, as Chapter 15 explained for learned Reward Models.

The group statistics are

$$\mu_r = \frac{1}{G} \sum_{j=1}^G r_j, \quad s_r = \sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \mu_r)^2}. \quad (3)$$

A robust group-relative normalization is

$$\hat{A}_i = \frac{r_i - \mu_r}{s_r + \delta}, \quad \delta > 0. \quad (4)$$

Here $\hat{A}_i$ is positive when response i scored above its group mean and negative when it scored below it. It is **relative reward information**: adding the same constant to every r_i leaves it unchanged. Dividing by the within-group spread makes the scale comparable across prompts whose raw reward magnitudes differ. The small δ is an implementation safeguard for zero or near-zero spread. The original GRPO formulation normalizes rewards by their group mean and standard deviation; precise choices for variance convention, clipping, reward transforms, and zero-variance handling vary across implementations [1].

Equation 4 is advantage-like, but it is not the learned value-based advantage $Q^{\pi(s_t, a_t)} - V^{\pi(s_t, a_t)}$ introduced in Chapter 16. It compares whole sampled responses for one prompt rather than predicting a return from every token prefix. In an outcome-reward implementation, the same $\hat{A}_i$ is often broadcast to every valid response-token position in y_i :

$$\hat{A}_{i,t} = \hat{A}_i, \quad 0 \leq t < T_i. \quad (5)$$

This gives all generated tokens in a successful response a shared positive signal and all tokens in a relatively poor response a shared negative signal. It is a coarse credit assignment, not evidence that every token causally produced the final outcome. More granular reward assignments exist, but they are outside this chapter’s scope.

2.1 Why a Group Can Replace a Separate Value Model

The expected return from a prompt can vary substantially with prompt difficulty, answer length, and reward scale. A learned critic in PPO attempts to predict that expected return from each state. GRPO replaces this prediction with evidence available at rollout time: for the same prompt, the sampled responses themselves estimate a local performance distribution. The group average acts as a baseline, so an update emphasizes responses that are better or worse than their siblings rather than merely responses with large raw scores.

This can eliminate the separate Value Model, its optimizer state, and its value-loss tuning. It does not eliminate the need for a baseline in the conceptual sense, nor does it create a precise token-level value estimate. Group-relative learning is most informative when the group contains meaningful variation. If every response is identically wrong, identically correct, or receives the same reward, then $s_r = 0$ and every numerator in Equation 4 is zero. Such a group supplies no ranking signal after the chosen zero-variance rule is applied.

3 The GRPO Policy Objective

Although GRPO changes the advantage estimator, it retains PPO’s local trust mechanism. For action $a_{i,t}$ sampled under π_{old} , define the token probability ratio

$$\rho_{i,t}(\theta) = \frac{\pi_{\theta}(a_{i,t} \mid x, a_{i,<t})}{\pi_{\text{old}}(a_{i,t} \mid x, a_{i,<t})}. \quad (6)$$

π_{old} is the rollout behavior policy, whereas π_{ref} below is the behavioral anchor. They have different jobs. The ratio in Equation 6 allows limited reuse of samples collected before the current update. The reference model constrains longer-term drift across many such rollout batches.

Let $\kappa_{i,t}(\theta)$ denote the declared KL penalty between the current and reference next-token distributions at the same prefix,

$$\kappa_{i,t}(\theta) = \text{KL}(\pi_{\theta}(\cdot \mid x, a_{i,<t}) \parallel \pi_{\text{ref}}(\cdot \mid x, a_{i,<t})). \quad (7)$$

One representative GRPO objective is

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=0}^{T_i-1} \left(\min(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_i) - \beta \kappa_{i,t}(\theta) \right) \right]. \quad (8)$$

The expectation in Equation 8 is over prompt batches and groups generated by π_{old} . The $\frac{1}{T_i}$ factor is one common response-length normalization; systems must declare whether they use it because changing the denominator changes how long responses are weighted. The objective is representative rather than universal. In particular, practical systems may use a sampled KL estimator, an alternative reduction over tokens, or a different reward-normalization convention.

For $\hat{A}_i > 0$, the clipped term gives the policy an incentive to increase the likelihood of the response’s sampled tokens, but only until the ratio exceeds $1 + \varepsilon$. For $\hat{A}_i < 0$, it discourages those tokens only until the ratio falls below $1 - \varepsilon$. This is the same local update-control idea as PPO’s clipped surrogate [2]. The reference penalty is separate: β controls the reward-versus-drift trade-off relative to π_{ref} , while ε restricts how far one update can move from π_{old} on one rollout batch.

3.1 Reward Normalization Is Part of the Objective

Group normalization changes the optimization signal rather than merely making logs look comparable. If raw rewards are shifted by a prompt-specific constant, Equation 4 is unchanged. If the reward spread is very small, normalization can magnify tiny reward differences unless the implementation explicitly guards against it. A discrete verifier can also produce a group with only zeros and ones; the resulting advantage depends on how many samples passed, not only on whether one individual response passed.

For this reason, reward transforms, reward clipping, standard-deviation convention, δ , and the treatment of all-equal groups are algorithmic choices. A system should not silently replace an all-equal group’s zero advantage with a global batch statistic and still call the result the same group-relative objective. Such a fallback may be useful, but it changes the baseline and should be evaluated as a different design.

4 Training Loop, Sampling, and Verifiable Rewards

The GRPO loop consumes fresh online data. Each prompt yields several responses under a fixed behavior-policy revision; scores and advantages are then frozen while the policy performs a limited number of clipped-update epochs. After that, the old group is discarded and a new group is sampled from the updated policy.

Stage	Operation	State that must remain attached
1	Sample prompts and freeze π_{old}	Prompt distribution, Chat Template, tokenizer, policy revision, and decoding settings.
2	Generate G Rollouts per prompt	Token IDs, stop reasons, masks, old-policy log probabilities, and response lengths.
3	Score every response	Reward-function revision, verifier or RM configuration, parsing result, and per-response reward components.
4	Normalize within each group	Group membership, mean, spread convention, δ , zero-variance policy, and advantages.
5	Run limited policy epochs	Reference revision, KL estimator, ε , β , valid-token denominator, optimizer state, and diagnostics.
6	Evaluate and resample	Independent evaluation, reward statistics, KL, pass rate, response length, and checkpoint identity.

Table 1: A GRPO iteration remains an on-policy dataflow. The group identity and the behavior-policy revision are part of the objective, not incidental batch metadata.

Group size G determines a direct statistical and systems trade-off. A larger group gives a more informative local ranking distribution and increases the chance that a difficult prompt contains both a strong and weak response. It also multiplies generation, reward-scoring, and memory or storage

cost per prompt. A small group is cheaper but may have unstable means and spreads; $G = 1$ gives no nonzero centered relative advantage at all. The useful group size depends on task difficulty, reward reliability, decoding diversity, and the available rollout budget rather than on a universal constant. GRPO is particularly natural when a response can be checked with a reproducible external rule. A mathematics answer can be compared with a known answer under a strict parser; a code solution can be run against tests; a symbolic derivation or formal proof can be checked by a declared system. These **verifiable rewards** can be sparse but comparatively reliable for the property they actually test. They do not certify every desirable property of the response: a parser can be gamed, a test suite can be incomplete, and a correct final answer does not necessarily validate every intermediate statement. The verifier, format convention, and failure behavior therefore define the operational reward function. A learned RM is also compatible with GRPO, but a group-relative advantage does not remove the RM’s bias, calibration, or distribution-shift limits described in Chapter 15. The relative construction can make a prompt-local comparison useful, yet a policy may still exploit features that raise the RM score without improving external quality. Independent evaluation remains necessary whether rewards come from a learned judge or an automatic checker.

5 GRPO, PPO, and DPO

GRPO and PPO are both online policy-optimization methods. They generate responses from a behavior policy, use an old-policy ratio, and commonly apply clipping and reference-model regularization. The difference is the advantage source. PPO learns or shares a critic that estimates value from token prefixes and can use GAE. GRPO obtains a response-level baseline from other samples for the same prompt and commonly avoids a separate Value Model. This reduces one substantial model and state burden, but it exchanges learned value prediction for extra rollout sampling and coarser outcome-level credit assignment.

DPO differs more fundamentally. It optimizes a fixed dataset of chosen–rejected pairs using policy and reference likelihoods; it does not require fresh Rollouts, a behavior-policy ratio, clipping against π_{old} , or group rewards. Its primary failure boundary is the coverage and quality of the offline preference corpus. GRPO can explore fresh policy behavior and consume a numeric reward from a verifier or RM, but it incurs the cost and instability controls of online generation. Neither method is automatically preferable: DPO offers a compact offline preference-learning interface, PPO offers flexible actor–critic optimization, and GRPO offers a group-relative alternative when multiple scored samples per prompt are feasible.

Property	GRPO	PPO and DPO contrast
Update data	Online groups of G Rollouts per prompt.	PPO uses online Rollouts; DPO uses offline chosen–rejected pairs.
Advantage or margin	Within-group normalized reward.	PPO uses a learned value-based estimate; DPO uses a reference-relative likelihood margin.
Auxiliary models	No separate Value Model in the basic formulation.	PPO normally has a critic; DPO needs neither a critic nor an explicit RM.
Feedback interface	Scalar scores from an RM, verifier, or other declared reward function.	PPO similarly accepts scores; DPO consumes pair orderings.
Main trade-off	Avoids critic state but requires multiple sampled responses.	PPO trades sampling for a critic; DPO trades online adaptation for offline simplicity.

Table 2: GRPO changes the online advantage estimator rather than replacing PPO’s trust mechanisms. DPO has a different, offline data interface.

6 Stability and Failure Modes

GRPO does not make policy optimization insensitive. A large learning rate, excessive reuse of one rollout group, broad clipping range, weak KL control, or a poorly specified valid-token denominator

can still produce abrupt policy changes. The diagnostics of Chapter 16 remain relevant: monitor objective terms, ratio and clip fractions, KL statistics, response length, entropy, finite values, and independent task results. Chapter 8’s numerical checks and Chapter 11’s checkpoint discipline also apply unchanged.

Reward variance has a special role. Groups with no reward variation provide no centered signal; groups whose scores are dominated by noise can create arbitrary positive and negative updates; groups with a single accidental high score can overemphasize an outlier. Increasing G may improve the chance of useful contrast, but it also costs more Rollouts and does not repair a biased reward function. Insufficient sampling diversity can make every response nearly identical, while excessive diversity can produce mostly invalid outputs whose comparisons teach little about the desired regime. The same reward-hacking and length-bias concerns recur in a new form. A policy can learn to exploit a verifier’s parser, a test-suite loophole, or an RM’s stylistic heuristic. A reward that favors long answers can alter both pass probability and group ranking; sequence-length normalization in the objective does not by itself establish a fair reward. Rapid KL growth can signal excessive movement away from the reference, whereas persistently negligible KL can indicate that rewards, advantages, or update settings are not producing useful learning. These are hypotheses to investigate with protected evaluation, not standalone diagnoses.

7 Implementation Contracts

The rollout contract must version the prompt distribution, Chat Template, tokenizer, policy and old-policy revisions, decoding parameters, group size G , maximum response length, stop-token policy, and response parser. It must preserve prompt identifiers, group membership, token IDs, response and action masks, old-policy log probabilities, stop reasons, and per-response lengths. A prompt token, padding token, and post-termination position must contribute to neither the policy objective nor its per-token KL term.

The reward contract must name the reward-function revision, whether the score comes from an RM, verifier, test executor, or mixture, and the serialization, parsing, timeout, failure, and reward-composition rules. It must retain the raw r_i , group mean, spread convention, δ , normalized $\hat{A}_i$, and all-equal-group policy. Tests should confirm that permuting group order only permutes the corresponding advantages; shifting every raw reward by a common constant leaves Equation 4 unchanged; and an all-equal group produces zero advantage under the declared fallback.

The optimization contract must distinguish π_{old} from π_{ref} , record the reference revision, exact KL estimator, ε , β , number of policy epochs, response-length reduction, optimizer and scheduler states, precision, effective batch size, gradient accumulation and clipping, and checkpoint policy. Logs should separately report raw reward, normalized advantage, verifier pass rate or RM score, reward spread, zero-variance-group rate, policy loss, ratio and clip fractions, KL, entropy, response length, throughput, and independent evaluation. Recoverable checkpoints require the policy, optimizer, scheduler, random states, prompt-data cursor, objective configuration, and any incomplete rollout-buffer state, following Chapter 11.

8 Summary

GRPO samples several responses for one prompt and turns their relative reward performance into an advantage-like training signal. Centering and scaling the group’s rewards makes a response positive when it is better than its peers and negative when it is worse. This prompt-local baseline can replace a learned Value Model when response-level comparisons provide enough information for the training task, but it does not create fine-grained credit assignment or remove the need for careful reward design.

The policy update remains PPO-like. A frozen behavior-policy snapshot supplies probability ratios, clipping limits reuse of a rollout batch, and a distinct reference model supplies KL control. The

resulting method is online: new policy behavior is generated, scored, normalized within groups, updated cautiously, and sampled again. The old policy and reference policy are different objects and must never be conflated.

GRPO is well suited to tasks with reproducible score functions, including mathematical answers, executable code, and symbolic checks, because multiple samples can expose prompt-local differences in correctness. Its limits are equally practical: group size raises sampling cost, equal or noisy rewards weaken learning, learned scores can still be hacked, and verifiers only reward what they actually check. GRPO, PPO, and DPO therefore represent different interfaces to feedback and compute, not a universal hierarchy of post-training methods.

References

- [1] Z. Shao *et al.*, “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024, doi: 10.48550/arXiv.2402.03300.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017, doi: 10.48550/arXiv.1707.06347.